

Days 17-21

Speculative decoding, tensor parallelism, the reconciliation pass over the repo, and two days of synthesis. The last block of the three-week plan. Same model and card: Qwen2.5-1.5B, fp16, one RTX 3060 12GB.

Day	Topic	The one thing you must be able to derive
17	Speculative decoding	Why it works at all, from your own step-time model
18	Tensor parallelism, and repo repair	Two all-reduces per layer, and why 2 KV heads caps TP at 2
19	Synthesis: the layer table	Concept, equation, and proving experiment, for every layer
20	Synthesis: the four anchors	All four, cold, timed, interrupted
21	Buffer and posture	What you are deliberately not learning, said confidently

The shift in these five days. Days 1 to 16 were acquisition. Days 17 to 21 are almost entirely retrieval and repair. Day 18 is the only day whose deliverable is a commit rather than a rehearsal, and it is there because an interviewer who opens the repo will find the inconsistencies before you get a chance to explain them.

Day 17 · Speculative decoding: buying tokens with spare arithmetic

This is a technique that was not run in this repo, which changes how it should be answered. Do not claim a measurement. Instead, derive why it works from the step-time model that *was* measured, and state the break-even condition. A derived projection with an explicit "I have not run this" is a much better answer than a recited paper summary.

17.1 The observation the whole technique rests on

From Day 13: a decode step costs $\tau_{\text{fixed}} + c_1 \times \text{batch}$, with τ_{fixed} around 11 ms and c_1 around 0.20 ms per sequence. τ_{fixed} is the shared weight read, 3.09 GB streamed to advance the sequence by **one** token. The tensor cores are idle through nearly all of it: arithmetic intensity around 1 against a ridge point of 142. Now ask the key question. If instead of one token, the same forward pass scored *six* candidate tokens, what would it cost? Almost the same, because the 3.09 GB weight read does not change and the extra arithmetic is free at intensity 1. The GPU has been doing nothing with its arithmetic for the entire step.

The one-sentence version. "Decode is memory-bound, so verifying k drafted tokens in a single forward pass costs about the same as generating one, since you re-use the same weight read. Speculative decoding spends the arithmetic that a memory-bound step was wasting anyway, and buys tokens with it."

17.2 The algorithm, in four steps

1. **Draft.** A small, fast model generates k candidate tokens autoregressively.
2. **Verify.** The large target model runs one forward pass over the prompt plus all k drafts, producing its own distribution at every one of those positions.
3. **Accept or reject.** Walk the drafts left to right. Accept a token with probability $\min(1, p_{\text{target}}(x) / p_{\text{draft}}(x))$. Stop at the first rejection.
4. **Correct.** At the rejection point, sample from the residual distribution $\text{norm}(\max(0, p_{\text{target}} - p_{\text{draft}}))$. If all k were accepted, sample one bonus token from the target's distribution at position $k+1$, which the verify pass already computed for free.

So a round produces between 1 and $k+1$ tokens, and never zero. That floor of 1 is what makes the technique safe: the worst case is a wasted draft, not a stall.

17.3 Why the output distribution is exactly unchanged

This is the property to be able to state precisely, and it is the reason speculative decoding is called lossless rather than approximate.

The accept rule plus the residual-sampling correction together produce samples from p_{target} exactly. Intuitively: accepting with probability $\min(1, p_{\text{target}}/p_{\text{draft}})$ keeps a token whenever the target likes it at

least as much as the draft did; where the draft over-proposed a token, the shortfall is exactly the mass $\max(0, p_{\text{target}} - p_{\text{draft}})$, and sampling the rejection from that renormalised residual puts the mass back where the target wanted it. Summed over both branches, the probability of emitting any given token is p_{target} . It is the same trick as rejection sampling with a proposal distribution, applied per position.

The consequence to say out loud: the draft model's quality affects *speed only*, never output quality. A bad draft model means a low acceptance rate and a slow run, not a worse answer. That is why speculative decoding is deployable in a way that, say, aggressive quantization is not.

17.4 The break-even condition, computed from this repo's own constants

Everything below is a **projection from measured constants**, not a measurement. Say so.

EXAMPLE 17-A: THE COST MODEL, BUILT FROM THE MEASURED T_FIXED AND C1

MEASURED, this repo:

$T_{\text{fixed}} = 11 \text{ ms}$ shared weight read, 3.09 GB of fp16 weights
 $c1 = 0.20 \text{ ms}$ marginal cost per extra sequence in the batch
 baseline batch-1 step = 11.28 ms/token (vLLM, RTX 3060)

PROJECTED draft cost. Qwen2.5-0.5B as the draft: 494M params = 0.99 GB fp16.

Decode is weight-read-bound, so scale T_{fixed} by the weight-byte ratio:

$T_{\text{draft}} = 11 \text{ ms} \times (0.99 / 3.09) = 3.5 \text{ ms per drafted token}$

PROJECTED verify cost. One forward pass over $k+1$ positions of ONE sequence, so the weight read is paid once and the extra positions cost roughly $c1$ each:

$T_{\text{verify}}(k) = 11 + 0.20 \times (k+1) \text{ ms}$

ROUND COST:

$T_{\text{round}}(k) = k \times 3.5 + T_{\text{verify}}(k)$

TOKENS PER ROUND, with per-token acceptance probability a (geometric):

$E[\text{tokens}] = (1 - a^{(k+1)}) / (1 - a)$

EXAMPLE 17-B: PROJECTED SPEEDUP AGAINST BASELINE, BY DRAFT LENGTH AND ACCEPTANCE RATE

	k=1	k=3	k=5	k=7
round cost:	14.7 ms	22.3 ms	29.7 ms	37.1 ms
$a = 0.5$	1.15x	0.95x	0.75x	0.60x
$a = 0.7$	1.30x	1.28x	1.12x	0.98x
$a = 0.8$	1.38x	1.49x	1.40x	1.26x
$a = 0.9$	1.46x	1.74x	1.78x	1.73x

read the table two ways:

DOWN a column: longer drafts only pay if acceptance is high.

ACROSS a row: the optimal k RISES with the acceptance rate.

$k=1$ at $a=0.5$, $k=3$ at $a=0.8$, $k=5$ at $a=0.9$.

and note the entries below 1.00x. at $a = 0.5$ with $k = 5$ speculative decoding is 25% SLOWER than just decoding, because 17.5 ms of drafting buys, on average, 1.97 tokens.

Break-even for $k = 5$: the round must yield $29.7 / 11.28 = 2.63$ tokens to be worth running at all. That is the number to quote. It converts a vague "acceptance rate matters" into a threshold.

17.5 Why the win is capped, and where it disappears

- **It is a latency technique, not a throughput technique.** It works by spending idle arithmetic. At batch 64 that arithmetic is no longer idle: from Day 13, the step is already at its skinny-GEMM ceiling and $c1 \times \text{batch}$ is 54% of the step. Speculation and batching compete for the same resource, and batching usually wins on a loaded server.

- **It costs KV memory.** The drafts occupy KV blocks during verification, and rejected ones must be rolled back. On a card that OOMs at 123,565 tokens, that is not free.
- **Acceptance rate is workload-dependent.** Code and structured text draft well because they are predictable; open-ended creative text drafts badly. A number measured on one workload does not transfer.

17.6 Medusa and EAGLE, one sentence each

Variant	The change	Why it is an improvement
Medusa	Delete the separate draft model. Bolt several extra decoding heads onto the target model itself, each predicting one position further ahead, and verify their combined candidates as a tree.	Removes the draft model's weight read entirely, which in the table above is the $k \times 3.5 \text{ ms}$ term, the dominant cost of a round.
EAGLE	Draft in <i>feature</i> space rather than token space: autoregress over the target model's second-to-top hidden states, which is a much easier prediction problem than predicting the token, and resolves to tokens only at the end.	Raises the acceptance rate, which the table shows is the variable that decides everything. Moving from $a = 0.7$ to $a = 0.9$ at $k = 5$ goes from 1.12x to 1.78x.

17.7 Self-test, Day 17

Q1. Why does verifying five tokens cost about the same as generating one?

Because batch-1 decode is memory-bound at an arithmetic intensity near 1 against a ridge of 142. The cost is the 3.09 GB weight read, which a single forward pass pays once no matter how many query positions it scores. On my measurements that read is τ_{fixed} , about 11 ms of an 11.28 ms step, and the extra positions cost roughly $c_1 = 0.20 \text{ ms}$ each.

Q2. Does speculative decoding change the model's output?

No, and that is the point of the accept-reject rule. Accept with probability $\min(1, p_{\text{target}}/p_{\text{draft}})$, and on rejection sample from the normalised residual $\max(0, p_{\text{target}} - p_{\text{draft}})$. The two branches sum to exactly p_{target} . The draft model's quality determines speed, never quality.

Q3. When would you not use it?

When acceptance is low or the server is loaded. On my step-time constants, $k=5$ with a 0.5 acceptance rate is 25% slower than plain decoding, because the round costs 29.7 ms and yields 1.97 tokens against a break-even of 2.63. And at high batch the idle arithmetic it exploits is already spoken for: at batch 64 more than half my step is per-sequence work, not the shared weight read.

Sources: Leviathan et al., Fast Inference via Speculative Decoding · Chen et al., Accelerating LLM decoding with speculative sampling · Cai et al., Medusa · Li et al., EAGLE

Day 18 · Tensor parallelism (45 minutes), then repo reconciliation

Two unequal halves. Tensor parallelism gets 45 minutes because there is one GPU here and a TP claim cannot be defended with anything but a paper. The rest of the day goes to fixing four places where a document has drifted from its own CSV, which is a real deliverable and a better research signal than one more topic.

18.1 Tensor parallelism: the split, and the two all-reduces

Split each layer's weights across GPUs so every device holds a slice and they cooperate on one forward pass. Megatron's arrangement, which everything since copies:

ATTENTION

split by HEAD. each GPU gets a subset of the query heads and their matching KV heads, computes attention for those heads only, and holds the corresponding COLUMN slice of o_proj.
-> one all-reduce after o_proj to sum the partial outputs.

MLP

gate_proj and up_proj are split COLUMN-wise (each GPU gets a slice of the 8960-wide intermediate). down_proj is split ROW-wise to match.
-> one all-reduce after down_proj.

TOTAL: 2 all-reduces per transformer layer, per token.
For this model: 28 layers x 2 = 56 collectives per decode step.

The model-specific constraint that makes this concrete. Qwen2.5-1.5B has 12 query heads and **2 KV heads** under GQA. Tensor parallelism splits by head, so the KV heads cap the degree: TP=2 gives each GPU 6 query heads and 1 KV head, which is clean. TP=4 cannot split 2 KV heads four ways, so it has to *replicate* them, which means every GPU stores the full KV cache for its replicated head and the memory saving TP was supposed to give partly evaporates. GQA and tensor parallelism interact, and this is the sentence that shows it.

18.2 Why it wants NVLink, in bytes and in microseconds

EXAMPLE 18-A: THE COLLECTIVE COST AT BATCH 1, TP=2

hidden = 1536, fp16. one all-reduce moves one hidden vector per token:
 $1536 \times 2 \text{ bytes} = 3072 \text{ B} = 3 \text{ KiB}$

per decode step: 56 all-reduces x 3 KiB = 168 KiB of traffic per token.

BYTES: 168 KiB over PCIe 4.0 x16 (~25 GB/s) = 6.9 us. nothing.

LATENCY: each collective costs a fixed launch-plus-sync of roughly 8 us over PCIe, or roughly 2 us over NVLink.

PCIe: $56 \times 8 \text{ us} = 0.45 \text{ ms}$ per token

NVLink: $56 \times 2 \text{ us} = 0.11 \text{ ms}$ per token

against a TP=2 step of about 5.5 ms (T_{fixed} halved), that is
7.6% overhead on PCIe, 2.0% on NVLink.

The conclusion is the interesting part: **decode-time tensor parallelism is latency-bound on the collective, not bandwidth-bound.** The payload is 3 KiB. What hurts is paying a synchronisation 56 times per token, on a step that is only a few milliseconds. That is why NVLink matters for TP and why TP degree does not scale gracefully over PCIe. Prefill is the opposite case, because the payload scales with sequence length and the bandwidth term starts to matter.

Projected TP=2 speedup on this model: T_{fixed} halves from 11 ms to about 5.5 ms, so the step goes 11.28 to roughly 5.95 ms including the PCIe collective cost. That is **1.9x**, **not 2x**, and the missing 0.1x is the 56 synchronisations.

18.3 Pipeline parallelism, one sentence

Split the model by *layer* rather than within a layer, so GPU 0 holds layers 0 to 13 and GPU 1 holds 14 to 27. It is a throughput technique, not a latency technique, because a single token still traverses every stage in sequence and the pipeline has bubbles at the start and end of every batch. Use it when you need to fit a model across nodes; use TP when you need the latency.

18.4 The reconciliation pass: four documents that disagree with their own data

These are live. An interviewer who opens the repo will hit them. Fixing them yourself and being able to say "I found four places where a doc had drifted from its CSV, and here is what I changed" is a better signal than any additional topic that could be crammed into this day.

#	File and what it says	What the data says	The fix
1	docs/oom-results.md, three places: "~60 KB/token", "roughly 2.15x the analytical 28"	results/oom_sweep.csv gives a slope of 62 KiB/token, 2.21x the analytical 28 KiB	Correct all three to 62 KiB/token and 2.21x, and use KiB rather than KB consistently, since the analytical figure is binary.
2	docs/baseline-hf-results.md line 22: decode "~18-23 tok/s", and the MBU paragraph built on 18 and 23	results/baseline_hf.csv settled at 28.05 to 28.32 tok/s across the last four fp16 runs. The 18.0 and 23.2 figures are two early runs before the transformers pin	Update the headline to ~28 tok/s, recompute the MBU line (28.1 tok/s = 35.5 ms/token = 87 GB/s = 24% of 360), and keep the 18/23 runs as an explicit note on run-to-run variance rather than as the result.
3	docs/baseline-vllm-results.md: decode 79.9 tok/s, headline "~4x". docs/batching-results.md: batch-1 decode 88.6 tok/s	Both are real, from different runs: 79.9 is the standalone baseline, 88.6 is batch 1 of the sweep under a different pool configuration and vLLM build	Either reconcile or caveat explicitly. The caveat is the honest option: state in both files which run each number came from and that they are 11% apart, rather than silently quoting whichever is convenient.
4	"~4x decode throughput" in three places in docs/baseline-vllm-results.md	$79.9 / 28.1 = \mathbf{2.84x}$ from the benchmark pair. results/profile_summary.csv gives 3.24x (40.65 to 12.56 ms per step) from the profiling run. Neither is 4x; the 4x came from dividing 79.9 by the stale ~20 tok/s	Settle on 3.2x as the headline, sourced from the profiling run, and say why: it is the matched comparison where both engines were measured in the same session with the same harness.

Why this is worth a whole day. Every one of these four is the same failure: a number was written into prose once and then the experiment was rerun. That is the most common way a measurement artifact rots, and noticing it is a research skill. The sentence to have ready is: "The CSVs are the record and the prose is a summary of them. Where they disagreed, I found four places, the CSV won, and I added a rule to the README saying so."

18.5 The rest of the reconciliation list

- **Write docs/quantization-results.md** so the NF4 result has a home matching the other three results docs. Content is Day 16: 2.68x smaller weights, 2.27x slower decode, the 62.3%-idle explanation, the bitsandbytes and sm_86 caveat, and the 30,489 tokens of extra context.
- **De-duplicate docs/phase1.md against weeks/week2.md**, which share paragraphs verbatim, including the scope statement and the directory tree.
- **Strip the stale T4 references.** docs/phase1.md still opens on "a single Colab T4" and its tree still lists a Colab runner notebook. The hardware of record is the Vast.ai RTX 3060. Keep the T4 mention only where it is deliberate: the fp16-not-bf16 rule, which exists for comparability with later V100 and T4 runs.

Sources: Shoenberger et al., Megatron-LM (tensor parallelism) · Narayanan et al., Efficient large-scale training on GPU clusters (PTD-P) · Ainslie et al., GQA (the head-count constraint)

Day 19 · Synthesis 1: concept, equation, proving result, for every layer

The table below is the artifact of this day, and it should be reproducible from a blank page. The discipline is the third column: **if a layer has no proving experiment or paper result, that is the layer to spend Day 21 on**. A concept without a proof attached is the thing that collapses under a follow-up question.

Layer	The concept, in one line	The equation	What proves it
1. Architecture	Attention mixes across positions; the MLP does not. GQA shrinks the KV cache by cutting KV heads, not query heads	$KV/token = 2 \times L \times H_{kv} \times d_{head} \times 2$	My config: 28 KiB/token under GQA against 168 KiB under MHA, a 6x ratio, from 2 KV heads against 12 query heads
2. Inference arithmetic	Prefill is compute-bound, decode is memory-bound, and the difference is arithmetic intensity, not model size	$ridge = FLOPS / bandwidth = 51e12/360e9 = 142 \text{ FLOP/byte}$	Measured 4205 tok/s prefill against 28.1 tok/s decode on the same model and card: a 150x per-token gap
3. KV cache	The KV cache is a growing tenant, and naive management wastes memory three ways: reservation, internal and external fragmentation	$peak = weights + 2 \times KV \text{ for a } torch.cat \text{ loop}$	OOM at 123,565 tokens; slope 62 KiB/token against 28 analytical; at the crash 1.3 GiB free inside the pool and unusable
4. GPU architecture	Kernels are limited by memory movement and occupancy far more often than by arithmetic	$MBU = bytes \text{ moved} / (time \times peak \text{ bandwidth})$	HF decode at 15-20% MBU; vLLM at 85% of the 291.5 GB/s I measured rather than the 360 GB/s spec
5. CUDA and the launch path	At small kernel sizes the dispatch chain above a kernel costs more than the kernel	$step = \sum(kernels) + \sum(gaps)$	HF: 1198 kernels per token, 62.3% device idle, 21.1 us of idle per launch against a 12.8 us average kernel
6. Landmark kernels	FlashAttention is an IO result, not a FLOPs result, and it does nothing at batch-1 decode	HBM traffic $O(N^2) \rightarrow O(N)$, compute unchanged at $O(N^2 \cdot d)$	Derivation, plus my own trace: attention is 0.61 ms of a 12.56 ms decode step, under 5%, and 28 flash split-KV kernels fire per step
7. Serving and scheduling	Batching converts a memory-bound step into a compute-bound one by amortising the shared weight read	$throughput = batch / (T_{fixed} + c1 \times batch)$, asymptote $1/c1$	88.6 to 2696 tok/s from batch 1 to 64, knee at 64, confirmed unmoved on an RTX 3090
8. Quantization	Weight-only quantization is a bandwidth play, so it only pays when the step is actually bandwidth-bound	$floor = weight \text{ bytes} / achievable \text{ bandwidth}$	My NF4 run: 2.68x smaller weights, 2.27x slower decode, because the baseline was 62.3% idle and dequantization added launches
9. Speculative decoding	Verifying k tokens costs about one step, so idle arithmetic buys tokens, at a rate acceptance decides	$break-even = T_{round} / T_{baseline} \text{ tokens per round}$	Projected from my constants: k=5 needs 2.63 tokens per round; at a=0.5 it is 0.75x, i.e. slower. Not measured, and I say so
10. Parallelism	TP splits within a layer for latency and costs two all-reduces per layer; PP splits across layers for throughput and has bubbles	56 collectives per token for 28 layers at TP=2	Paper-level only. My card count is one. The concrete hook I do have is that 2 KV heads cap TP at 2 for this model

19.1 The two rows to be honest about

Rows 9 and 10 are the only ones whose third column is not a measurement from this repo, and the correct move is to say that before being asked. "Everything in this table except speculative decoding and parallelism is something I measured on my own hardware. Those two I can derive and place, but I have one GPU and I did not run them." Volunteering the boundary of your evidence is what makes the rest of the table credible.

19.2 The single thread that connects all ten rows

Say this if asked "what did you actually learn from all this?" "That at batch 1 on a small model, almost nothing that limits you is arithmetic. My decode step was 62.3% idle, ran at 15-20% of memory bandwidth, and did 3.1 GFLOP against a card capable of 51 TFLOPS. Every optimization that helped attacked either bytes moved or time not spent computing: CUDA graphs removed the idle, fused kernels cut 1198 launches to 356, batching amortised the weight read, PagedAttention removed the fragmentation. And the one optimization that attacked bytes when bytes were not the constraint, my 4-bit run, made things 2.27x worse. That ordering is the whole lesson."

Day 20 · Synthesis 2: the four anchor answers, cold and timed

Almost any question routes back to one of these four. Each should take 60 to 90 seconds out loud. Deliver each one three times today: once from notes, once from a blank page, once interrupted.

20.1 Anchor 1: the FlashAttention derivation

The only anchor with no supporting number in this repo, so the derivation has to carry it.

- BEAT 1 standard attention writes the full $N \times N$ score matrix to HBM, reads it back for softmax, writes it, reads it again for the V multiply. the FLOPs are unavoidable. the round trips are not.
- BEAT 2 the obstacle: softmax needs the whole row for its denominator, which looks incompatible with tiling.
- BEAT 3 online softmax fixes it. carry m, l, O . when a tile arrives with a bigger max, rescale by $c = \exp(m_{\text{old}} - m_{\text{new}})$ and continue. exact, not approximate. it is the same max-subtraction you already need for numerical stability, turned into an incremental algorithm.
- BEAT 4 so FlashAttention tiles Q, K, V into SRAM and never materialises the score matrix. HBM traffic falls from $O(N^2)$ to $O(N)$; compute is unchanged, and the backward pass does MORE arithmetic by recomputing. it is an IO-complexity result.
- BEAT 5 the inference caveat, which is what shows you thought about decode: at batch 1 the score matrix is $1 \times n$. no quadratic term to avoid, no tile worth keeping. decode's problem is the opposite, not enough parallel work. the answer is flash-decoding / split-KV, which splits the KV sequence across SMs and merges partials with the same rescale. in my own vLLM trace: 28 split-KV kernels and 28 combine kernels per step, one pair per layer, at 14.9 to 16.2 us each.

20.2 Anchor 2: the KV-cache OOM

SETUP	I wanted to watch the KV cache kill a GPU rather than compute that it would. Qwen2.5-1.5B fp16 on a 12GB RTX 3060, 32-token prompt, context grown purely by decoding one token at a time, VRAM logged every 1000. six hours of sequential decode. it died at 123,565 tokens.
MATH	analytical: $2 \times 28 \text{ layers} \times 2 \text{ KV heads} \times 128 \text{ head_dim} \times 2 \text{ bytes}$ = 28,672 B = 28 KiB/token. the 2 KV heads size the cache, not the 12 query heads. that is GQA.
GAP	measured slope was 62 KiB/token, 2.21x the prediction, and sat above it the whole way. that gap is a MECHANISM, not overhead: HuggingFace reallocates the whole cache every step. torch.cat cannot grow a tensor in place, so it allocates a fresh block for n+1 tokens and copies n in, and both copies are live during the copy. peak is weights + 2 x KV. at 120k that predicts 9507 MiB; I measured 10,227, the ~700 MiB residual being attention scratch that also grows with n.
CRASH	and the death is FRAGMENTATION, not exhaustion. at OOM: 10,437 MiB allocated, 11,764 MiB reserved. 1.3 GiB free inside PyTorch's own pool and unusable, because the next cat needed one contiguous 2 x KV slab and the free space was scattered between parked blocks. enough empty parking spaces for the bus, no unbroken stretch to fit it.
OWN IT	I predicted the card would die near 30k tokens and kept the wrong guess in the writeup. it reached 123k. the mechanism I named was right and my arithmetic was pessimistic.
LAND	both mechanisms are exactly what PagedAttention removes: fixed-size blocks with a block table, so there is no cat, no second copy, and any freed block fits any future need. that is the paper's "external fragmentation" category, and I watched it before I read the name.

20.3 Anchor 3: the batching knee

WHY	batch 1 wastes the GPU: the step reads all 3.09 GB of weights to advance one sequence by one token. intensity ~1, tensor cores idle. so I swept vLLM concurrency 1 to 256 on the same 512-token prompt.
WHAT	decode throughput 88.6 -> 2696 tok/s from batch 1 to 64, a 30x gain, then essentially stopped: 2967 at batch 256, while TPOT climbed 11.28 -> 86.29 ms. the knee is 64.
MODEL	step = $T_{\text{fixed}} + c1 \times \text{batch}$. $T_{\text{fixed}} \sim 11 \text{ ms}$ is the shared weight read paid once regardless of batch; $c1 \sim 0.20 \text{ ms}$ is what each added sequence costs. throughput = $\text{batch} / (T_{\text{fixed}} + c1 \times \text{batch})$: linear while the fixed cost dominates, asymptotic to $1/c1 \sim 3000$ once it does not. knee where they cross: ~58 predicted, 64 measured.
TWIST	the plateau is not a roofline. at batch 256 I am at ~9 TFLOPS (16% of ~51) and ~90 GB/s (25% of 360). neither is saturated. the limit is operation shape: batched decode is a skinny GEMM with $M = \text{batch}$, occupancy-bound, and the paged KV reads are scattered. the naive roofline predicts an ideal batch of 142 against a measured 64, and that 2x-4x gap is itself the result.
TEST	I made it falsifiable. I predicted the knee is a model property, not a card property, because it is the weight-read-versus-KV-read crossover. on an RTX 3090, 2.6x bandwidth and 1.4x compute, every throughput number scaled 2.4x-3.4x and the knee stayed at 64, TPOT flat through 64 and breaking at 96 on both cards. I got the plateau HEIGHT wrong: predicted 6-7k from bandwidth alone, measured 10,100, because the 3090's extra compute also lifts the skinny-GEMM ceiling.

20.4 Anchor 4: the HF-versus-vLLM profiling result

CLAIM	vLLM decodes ~3.2x faster than HuggingFace at batch 1, same card, same model. the usual explanation is "CUDA graphs and fused kernels", which is an argument, not a measurement. so I profiled it as a 2x2 ablation over torch.compile and CUDA-graph capture, plus HF.
HEAD	of the 28.09 ms per token separating them, 25.34 ms (90%) is device idle removed and 2.60 ms (9%) is faster device work. the gap is a SCHEDULING problem, and now there is a number behind that sentence. HF fires 1198 kernels with 1198 CPU launches and is 62.3% idle. per-launch idle is only 21.1 us against a 12.8 us average kernel: it is not that a launch is expensive, it is that it is paid 1198 times, and batch-1 decode kernels are microsecond matrix-vector products, so the dispatch chain above a kernel costs more than the kernel.
SPLIT	the ablation separated two things the usual story conflates. FUSION is vLLM's hand-written kernels, before any compiler runs: with compile and graphs off on both sides, kernel count falls 1198 -> 356, a 3.4x cut. fused RMSNorm, fused rotary, fused SiLU-and-multiply, paged FlashAttention. CUDA GRAPHS change no kernels at all: counts are identical with graphs on and off, but CPU launches fall 356 -> 17 and idle goes to zero. I stated the prediction first: if graphs remove idle and nothing else, the graphs-on step should equal the graphs-off busy time. predicted 13.02 and 12.90 ms, measured 12.57 and 12.56. within 3.5% and 2.6%.
BEST	the result I did not predict is the most useful: the two are NOT additive. torch.compile saves 2.81 ms with graphs off and 0.01 ms with graphs on. once graphs take the CPU off the critical path, making the CPU's per-launch work cheaper is worth nothing. measuring only the shipping default against plain eager would have credited graphs with a fusion win, or the reverse, with no way to tell.
NEXT	and it closes the question of what is left. against this card's MEASURED achievable read bandwidth of 291.5 GB/s, not the 360 GB/s spec figure, vLLM's decode step runs at 85%. there is almost nothing left to win from scheduling at batch 1. the next lever has to be arithmetic intensity, which is the batching experiment.

20.5 The routing table: which anchor answers which question

If asked	Route to
"Walk me through a memory problem you debugged"	Anchor 2, in full
"Explain FlashAttention" / "why is attention slow"	Anchor 1, in full
"How would you pick a batch size" / "what is throughput vs latency"	Anchor 3, then Day 13's goodput and TTFT argument
"Why is vLLM faster than transformers"	Anchor 4, in full
"What is PagedAttention"	Anchor 2's crash, then the block table as its removal
"Why is decode memory-bound"	Anchor 3's opening, then the 150x prefill-decode gap
"Tell me about quantization"	Day 16's NF4 result, which routes into Anchor 4 for the 62.3%-idle explanation
"What would you do next on this work"	Anchor 4's closing: scheduling is at 85% of achievable bandwidth, so the remaining lever is arithmetic intensity

Day 21 · Buffer, and what you are deliberately not learning

21.1 Use the buffer on the gap Day 19 exposed

Day 19's table had two rows whose third column was not a measurement, and probably one or two more whose second column could not be reproduced cold. Spend the buffer there. Do not spend it on a new topic. Three weeks buys depth on ten layers or a mention of twenty, and the second option loses every follow-up question.

21.2 The deprioritization list, said confidently

Naming these is a positive signal. "Aware of it, have not gone deep" is a correct and well-calibrated answer. Hedging or bluffing is not.

Cut	The reason, in the form to say it
MoE routing and expert parallelism	"Single-GPU dense-model artifact. Everything I measured is about weight-read amortisation, which MoE changes qualitatively. I would rather be precise about dense inference than vague about both."
Ring attention, sequence parallelism	"Multi-GPU long-context technique. I have one card, and my long-context result is a memory result, not a distribution result."
FP8 and FlashAttention-3	"Hopper-class features. My card is Ampere sm_86. I could recite them but not measure them, and reciting is the failure mode I am avoiding."
MLA beyond one sentence	"One sentence is enough: it compresses K and V into a shared low-rank latent, so it attacks the same KV-bytes term as GQA but harder. I can place it on the MHA-MQA-GQA axis; I have not derived it."
P/D disaggregation (DistServe, Mooncake)	"Multi-node serving architecture. I understand the motivation directly from my own data, since prefill is compute-bound at 4205 tok/s and decode is memory-bound at 28, so co-locating them means one phase always interferes with the other. I have not studied the systems."
Structured decoding, multi-LoRA	"Serving features orthogonal to the performance question this artifact is about."
Training-side anything: ZeRO, FSDP, activation checkpointing	"Out of scope by construction. Inference has no backward pass, no optimizer state and no gradient memory, which is exactly why the KV cache is the dominant growing tenant rather than activations."
Writing a production CUDA kernel	"Three weeks buys diagnosis, not authorship. I can read a kernel and tell you why it is slow in coalescing, occupancy and shared-memory terms. Claiming authorship I cannot demonstrate is worse than claiming neither."

21.3 The habit that decides whether any of this worked

End every topic by explaining it out loud, unaided, in under 90 seconds. If you cannot, re-derive it from scratch rather than rereading. Rereading produces recognition; re-derivation produces defence, and only one of those survives being interrupted with "wait, why is it two KV heads and not twelve?"

The measure of a day is not what you read. It is what you can now say cold.

21.4 Final checklist, all three weeks

1. Whiteboard attention shapes through one decode step of this config, unaided.
2. State the ridge point (142 FLOP/byte), where decode sits (intensity ~1), and where prefill sits.
3. Hand over two versions of a matmul kernel and explain which is faster, in coalescing, shared-versus-global and occupancy terms, with numbers.
4. Derive the online-softmax rescale on a blank page and verify it against a direct computation.
5. Say why FlashAttention does nothing at batch-1 decode, and what does instead.
6. Tell the OOM story in under 90 seconds with four numbers in it.
7. Name both batching plateaus, their values, and the distinct mechanism behind each.
8. Give the idle-versus-kernel split of the vLLM win, and why the two optimizations are not additive.
9. Explain why your 4-bit run got slower, and name the two assumptions that would have to change.
10. Deliver all four anchor answers cold, timed, and interrupted.

The last thing to remember. The strongest material in this repo is not the speedups. It is the three places where a prediction failed and was diagnosed: the OOM cliff came at 123k tokens rather than the 30k predicted, the 3090 plateau came in at 10,100 tok/s rather than the 6-7k predicted, and the 4-bit run got slower rather than faster. Lead with those when asked what you learned. Anyone can report a number that confirmed what they expected.

Sources: [Weng, Large Transformer Model Inference Optimization](#) · [gpt-fast](#) (about 1000 lines that do everything the profiling result says matters) · [Pope et al., Efficiently Scaling Transformer Inference](#) · [Zheng et al., SGLang / RadixAttention](#)

Repo sources for every measured number above: `results/baseline_hf.csv`, `results/baseline_vllm.csv`, `results/profile_summary.csv`, `results/oom_sweep.csv`, `results/vllm_batch_sweep.csv`, `results/trace_vllm-graph.json.gz`, `docs/profiling.md`, `docs/batching-results.md`, `docs/oom-results.md`. The speculative-decoding figures in Day 17 and the tensor-parallelism figures in Day 18 are **projections** from the measured `Tfixed` and `c1` constants, not measurements, and are labelled as such where they appear. The four discrepancies listed in Day 18 are live in the repo as of this writing; the day's deliverable is the commit that closes them. Where a CSV and a prose doc disagree, the CSV is used.